

Swiss Map in Go 1.24

Vladimir Saraikin



Introduction

About me:

- Software developer for several trading/finance companies

Today's talk:

- What is a hash table and how Go's map worked before 1.24
- What changed with Swiss Tables and why it's faster
- Benchmarks and tradeoffs

Why Should We Care?

Map operations as % of total CPU time:

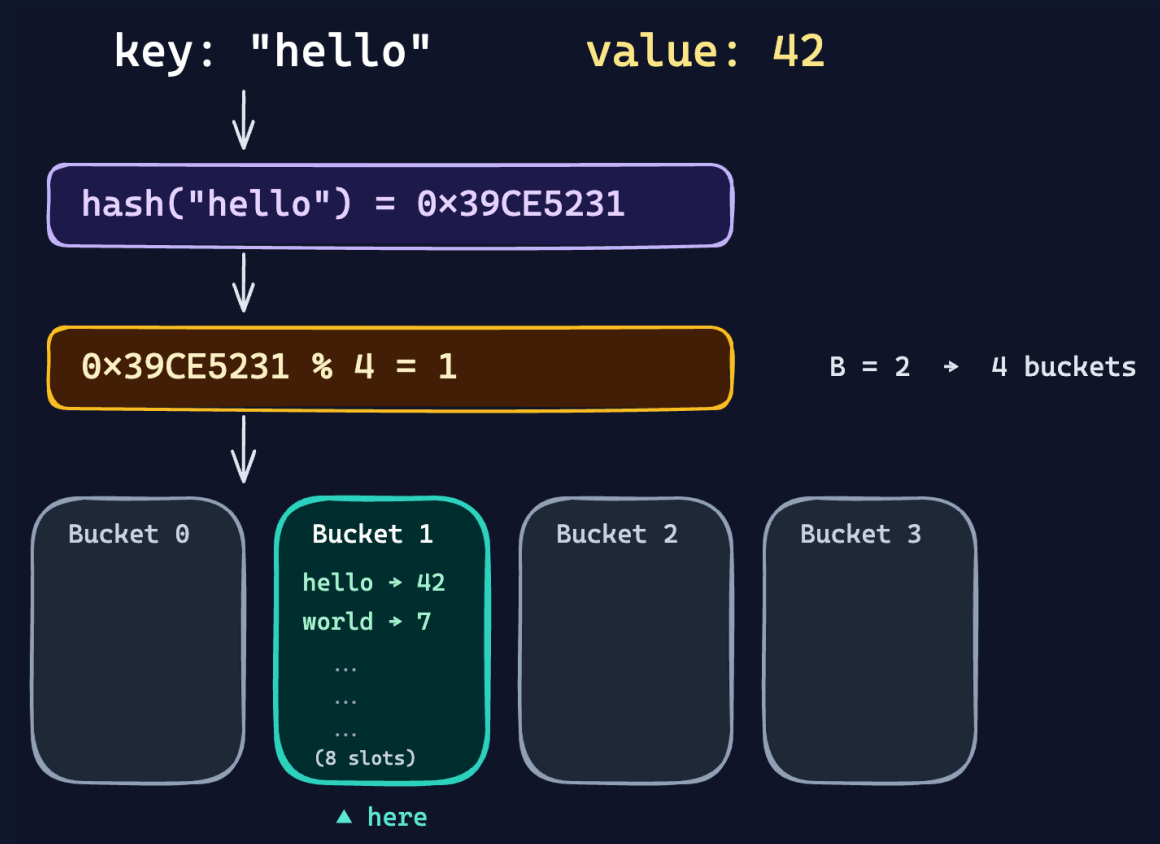
- **Google fleet** — 10-15%
- **ByteDance** — ~4%
- **Go 1.24 savings** — 2-3% fleet-wide

Origin Story

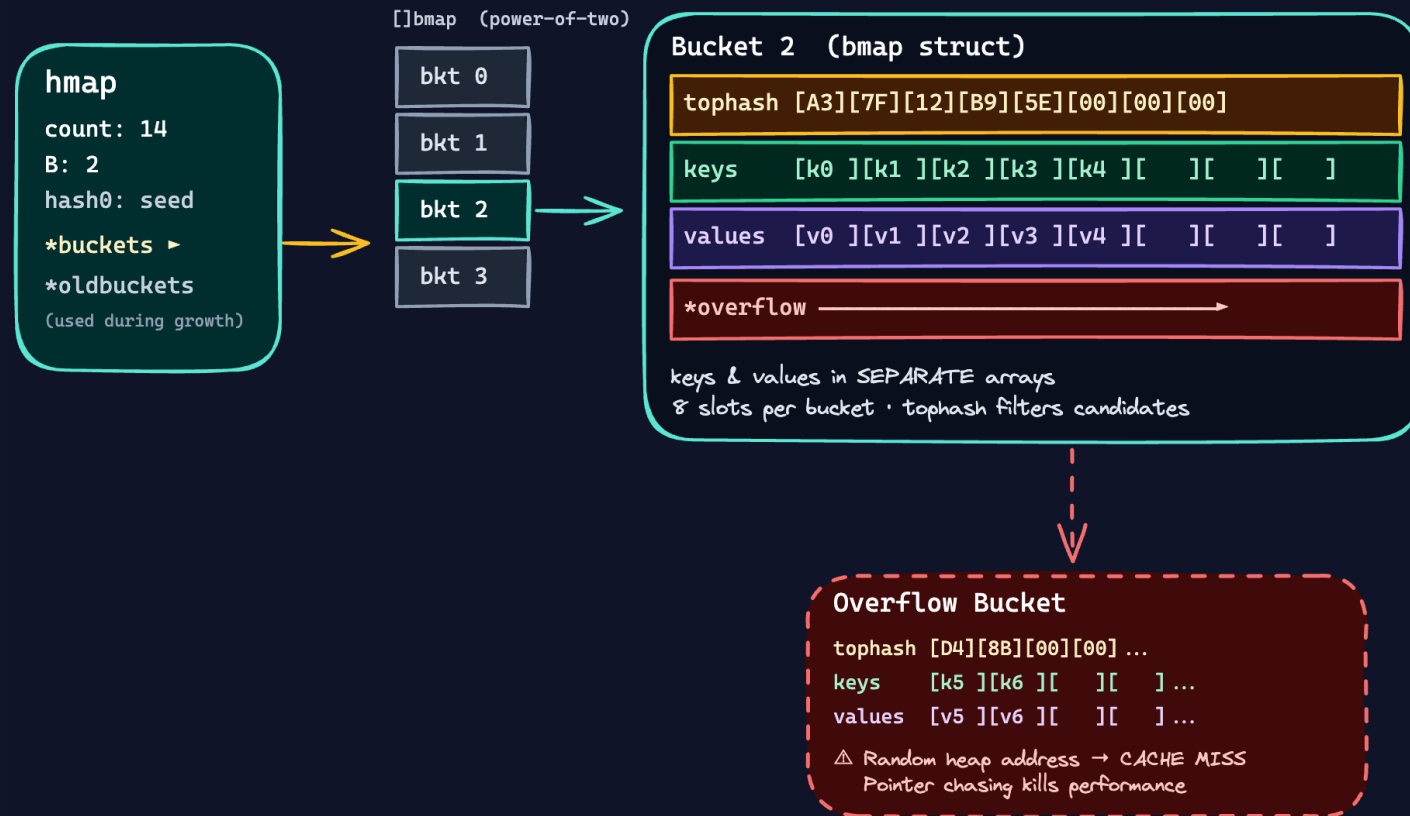
- **2017** — Google Zurich, Swiss Tables for C++
- **2018** — open-sourced in Abseil
- **2022** — Go proposal #54766
- **2025** — Go 1.24 ships with new map

Part I: The Old Map

Hash Table: The Idea

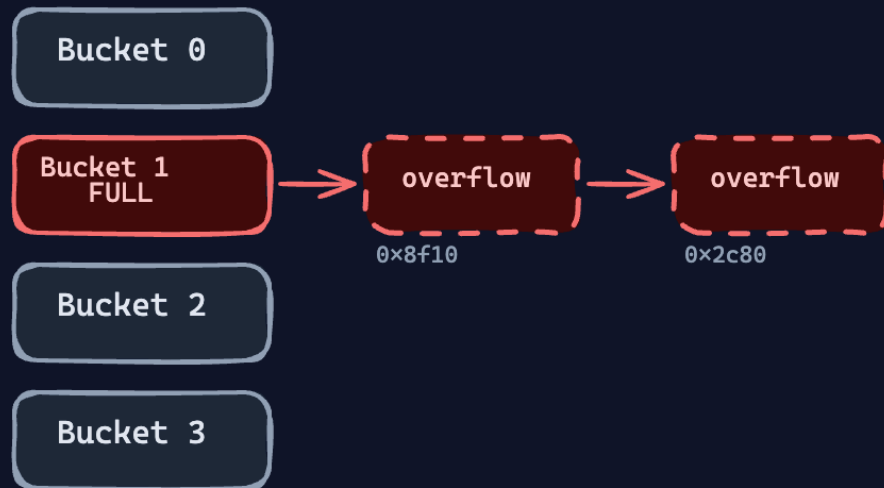


Old Map: Structure



Collision Resolution: Chaining

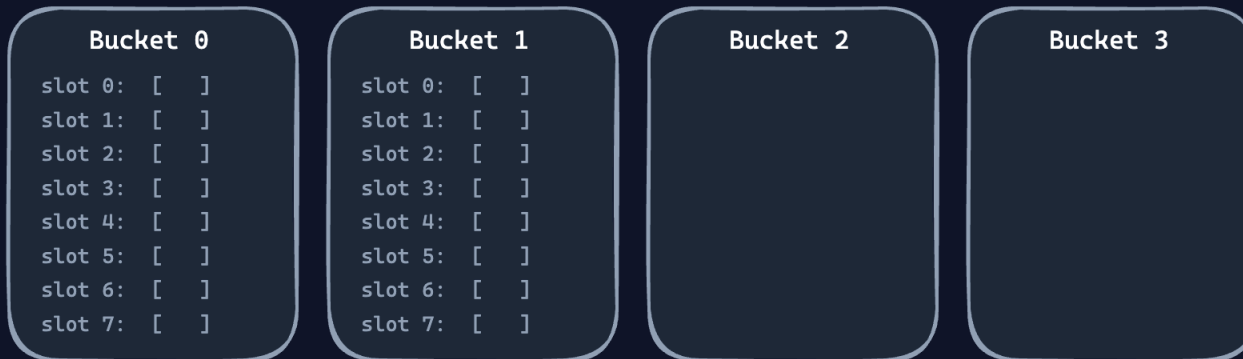
Separate Chaining



- X Overflow buckets at random heap addresses
- X Each hop → pointer chase → cache miss (~100 cycles)
- X Extra allocations → GC pressure

Old Map: Why 4 Buckets?

$B = 2 \rightarrow \text{numBuckets} = 2^2 = 4$



4 buckets × 8 slots each = 32 total

Don't confuse:

4 = number of buckets (controlled by B)

8 = slots per bucket (hardcoded in Go)

Why power of 2?

$\text{hash} \% n \rightarrow \text{hash} \& (n-1)$

One AND instruction instead of expensive division

Old Map: How Many Buckets?

How many buckets? B controls everything

`numBuckets = 2^B` `loadFactor = 6.5/bucket` B grows with data:

`make(map[string]string, 10):`

10 / 6.5 = 1.54 buckets minimum

nearest power of 2 $\geq 2 \rightarrow B = 1$

$2^1 = 2 \text{ buckets} \times 8 \text{ slots} = 16 \text{ capacity}$

B=0 1 bucket 8 slots

B=1 2 buckets 16 slots

B=2 4 buckets 32 slots

B=3 8 buckets 64 slots

...

When to grow?

avg elements per bucket > 6.5 $\rightarrow B++ \rightarrow 2x \text{ buckets} \rightarrow \text{evacuation}$

Why power of 2?

`hash % numBuckets` $\rightarrow$ `hash & (numBuckets - 1)` // 1 AND vs expensive division

Old Map: Hash → Bucket



```
bucketIndex := hash & (numBuckets - 1)
```

Old Map: Inside a Bucket

Bucket 1 (bmap struct)

tophash:

A3	7F	12	B9	5E	00	00	00
----	----	----	----	----	----	----	----

top 8 bits of hash → quick filter before comparing full key

keys:

k0	k1	k2	k3	k4			
----	----	----	----	----	--	--	--

all keys stored in one contiguous array

values:

v0	v1	v2	v3	v4			
----	----	----	----	----	--	--	--

all values stored in SEPARATE array (not interleaved with keys)

***overflow:** nil (→ points to next bucket when full)

when all 8 slots filled → new bucket allocated on heap → linked via this pointer

Lookup flow:

1. hash(key) → top 8 bits
2. scan tophash[0..7]
(one by one, sequentially)
3. tophash match?
→ compare full key (expensive)
4. not found in 8 slots?
→ follow *overflow pointer
→ CACHE MISS → repeat

Old Map: 9th Element → Overflow

Bucket 1 (FULL)

tophash [A3][7F][12][B9][5E][C1][3A][D4]
keys [k0][k1][k2][k3][k4][k5][k6][k7]
values [v0][v1][v2][v3][v4][v5][v6][v7]

*overflow: 0x8f10 ▶

9th element → no space!

▼ allocate on heap

Overflow Bucket

addr: 0x8f10 (random heap!)

tophash [F1][00][00][00][00][00][00][00]
keys [k8][][][][][][][][]
values [v8][][][][][][][][]

*overflow: nil

(more overflow? another hop...)

- ⚠ Each overflow = new heap alloc
- Random address = CPU cache miss
- Chain grows = $O(n)$ worst case

Old Map: Tophash & CPU Cache Lines

Sequential Compare + Cache Lines

CPU Cache Line (64 bytes):

[k0] [k1] [k2] [k3] [k4] [k5] [k6] [k7] ← all loaded at once!

1. Compare k0 = key?

x no

★ OS context switch!
Another process uses CPU registers.
Your cache line gets EVICTED.

2. Compare k1 = key?

← must RELOAD from RAM (~100 cycles)

3. Compare k2 = key?

... x 8

Swiss Table solution:

1 SIMD instruction → compare ALL 8 at once → no window for eviction

Old Map: Evacuation

Before evacuation:

```
hmap
count: 26
B: 2 (4 bkts)
*buckets ▶
*oldbuckets: nil
avg 6.5/bkt = 81%
```

```
*buckets [bkt0][bkt1][bkt2][bkt3]
```

4 buckets, nearly full with overflow chains

Load factor > 6.5 per bucket!
Evacuation triggered on next insert



During evacuation:

```
hmap
B: 3 (8 bkts)
*buckets ▶
*oldbuckets ▶
BOTH arrays
live in memory!
```

```
*buckets (NEW) [bkt0][bkt1][bkt2][bkt3][bkt4][bkt5][bkt6][bkt7]
```

```
*oldbuckets [bkt0][bkt1][bkt2][bkt3]
```

↑ data migrates
| incrementally per insert

⚠ 2x memory peak: both arrays live until migration completes
Every lookup checks BOTH arrays during this window

Old Map: Summary of Problems

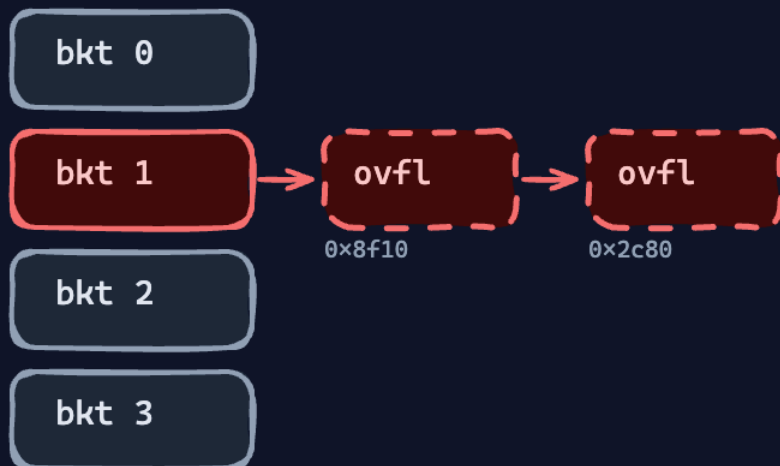
#	Problem	Why it hurts
1	Overflow chains	Pointer chasing → cache miss per hop
2	Sequential compare	8 separate ops → context switch evicts cache
3	Expensive growth	Evacuation copies all data, 2× memory peak
4	Low load factor	81% → wastes memory, grows too early
5	Separate K/V arrays	Poor locality within bucket

Part II: Swiss Table

Collision Resolution: Chaining vs Open Addressing

Separate Chaining (1953)

Old Go map used this



- X Random heap addresses
- X Pointer chasing → cache miss
- X Extra allocations → GC pressure

Open Addressing (1957)

Swiss Table uses this



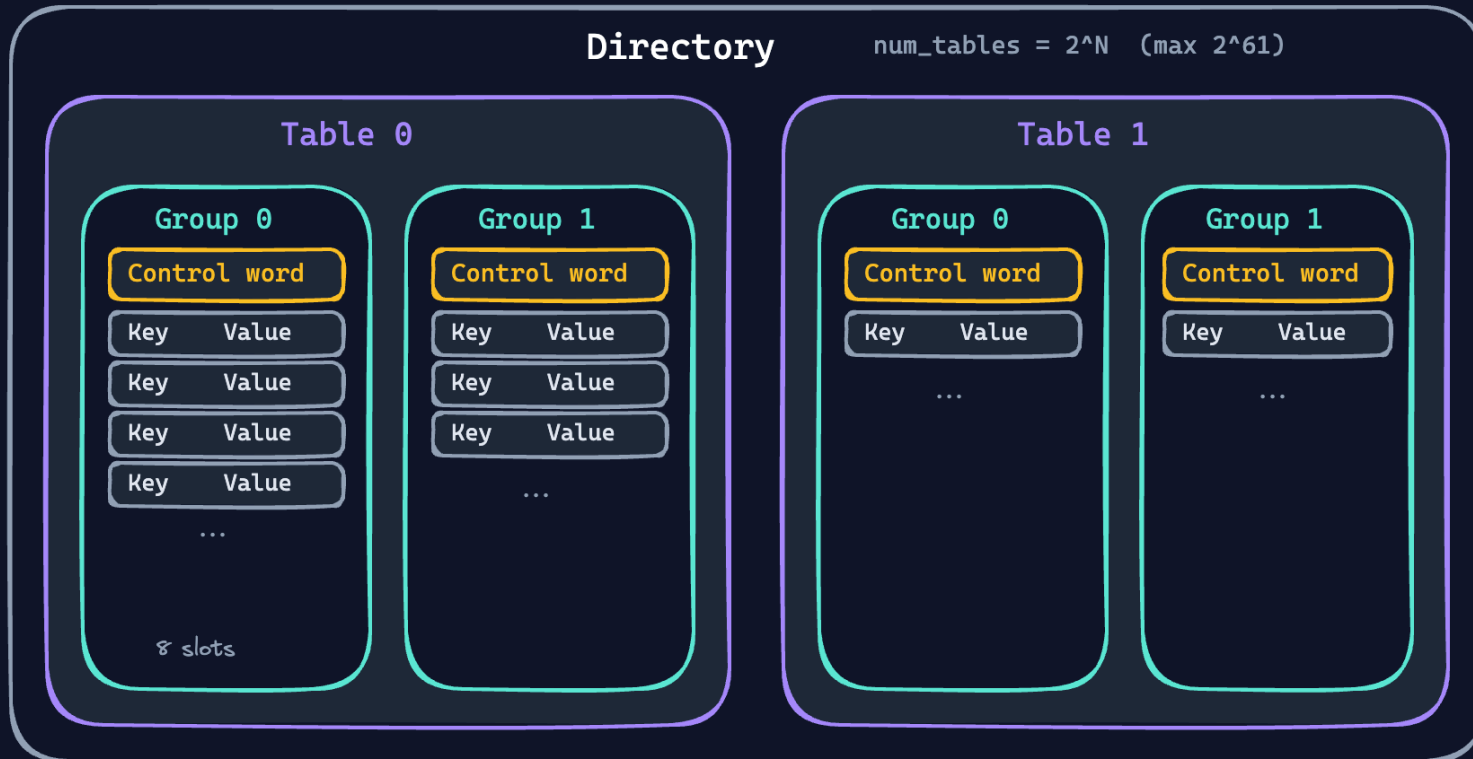
k5 can't go to slot 0 → probed to slot 2

ONE contiguous array
All data in sequential memory addresses

- ✓ CPU prefetcher works
- ✓ No pointer chasing
- ✓ No extra allocations

- X Delete needs tombstones
(can't just clear a slot — breaks probing)

Swiss Map Structure



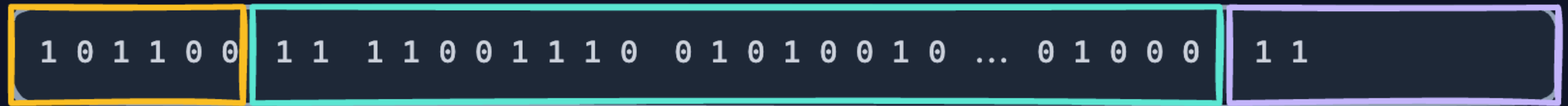
Directory = manages tables

Table = up to 1024 groups

Group = 8 slots + control word

Hash Split: h1 and h2

hash(key) → 64-bit number:



dir index

N bits (variable)



select TABLE
hash >> (64 - N)
2-3 instructions

h1

51 bits



select GROUP
h1 & (numGroups - 1)
1 AND instruction

h2

7 bits



stored in CTRL WORD
for parallel matching
(the M byte)

Metadata: h2 + Occupancy Bit

1 byte per slot = metadata (M)

7 bits = h2

1 bit = occupied?



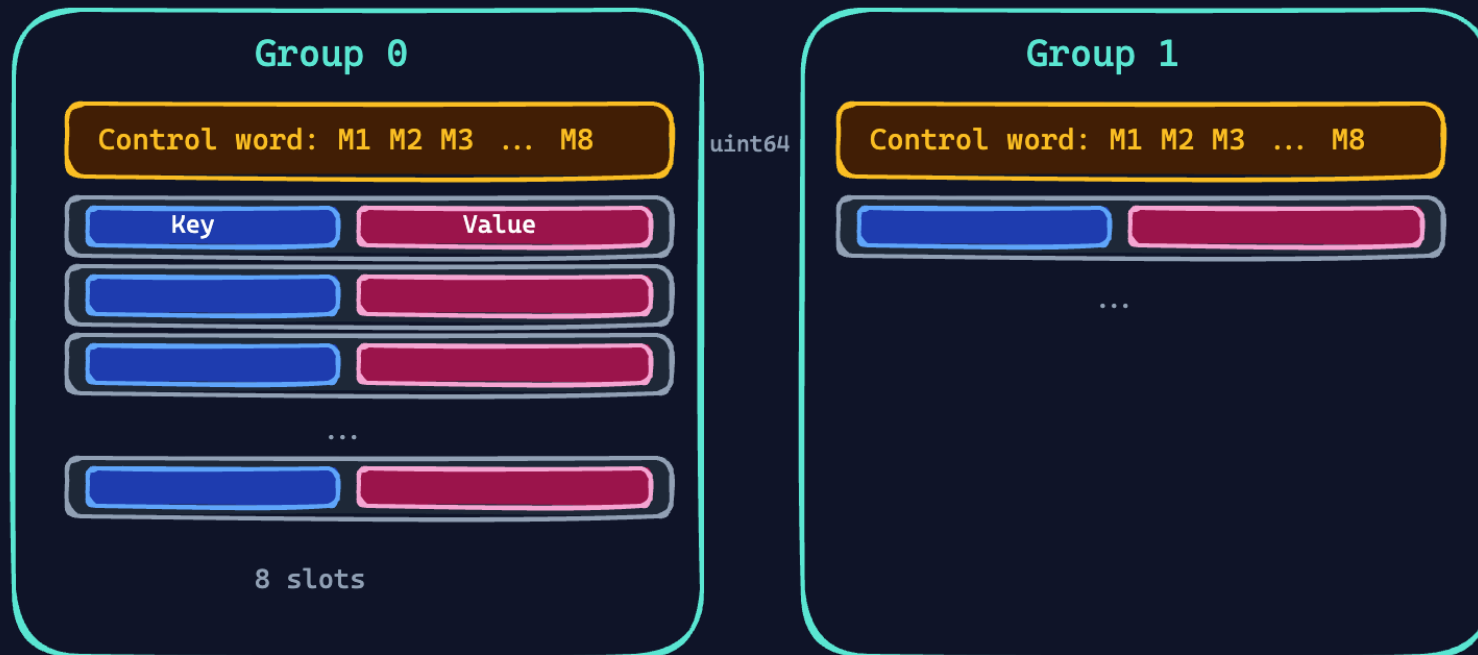
← occupied (h2 stored)

← empty (0x80 = high bit set, slot free)

8 slots per group → 8 bytes total → fits in one cache line

Group: The Unit of Search

Go uses 8 slots per group



Contiguous memory → no pointer chasing, CPU prefetcher works

Slot Internals

Slot layout: $K + V$ together

Value ≤ 128 bytes: inline



← same cache line, great locality

Value > 128 bytes: pointer in slot



Old map: `keys[]` and `values[]` in separate arrays

Swiss: `[K|V] [K|V] [K|V] ...` together in slot

Parallel Matching: The Idea

Looking for $h_2 = 0 \times 23$

Control word:

23	89	50	80	80	80	80	80
----	----	----	----	----	----	----	----

compare all 8 at once

Replicated h_2 :

23	23	23	23
----	----	----	----

 ... $\times 8$

Result mask:

1	0	0	0
---	---	---	---

 ... $\times 8$

↑

match! → check full key in slot 0

Parallel Matching: How It Works

Portable path: no SIMD, pure math

1. Broadcast h2

ctrl: 23 89 50 80 80 80 80 80

h2x0x01..01 = 23 23 23 23 23 23 23 23

2. XOR

ctrl ^ replicated = 00 AA 73 A3 A3 A3 A3 A3 match → 0x00

3. Detect zeroes

bit trick → bitmask: 80 00 00 00 00 00 00 00 slot 0 = match!

5-6 instructions. No loops, no branches.

Platform	Path	Instructions
x86 + GOAMD64=v2	SIMD	3 (~1 cycle)
x86 default (v1)	Portable	5-6
ARM (Apple Silicon)	Portable	5-6

Insert: Hash the Key

Step 1: $\text{hash}(\text{key}) \rightarrow h1 + h2$

`hash64("mykey") = 0x3A7F ... B423`



`h1 (upper 57 bits)`
→ selects group

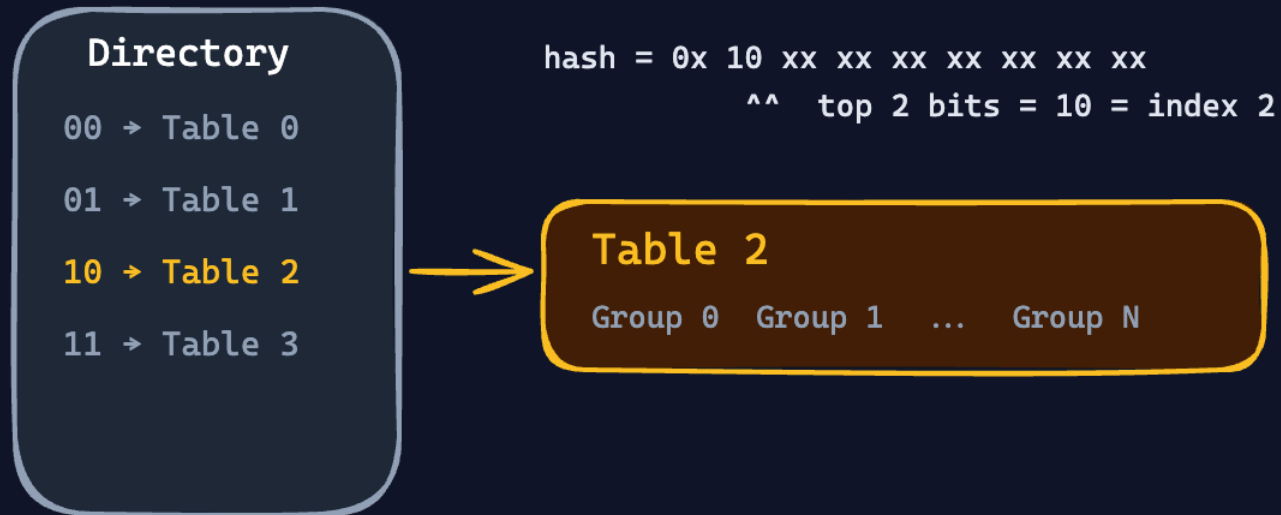
`h2 (lower 7 bits)`
→ fast matching

Insert: Select Table

Step 2: h1 selects table from directory

```
table_index = hash >> (64 - global_depth)
```

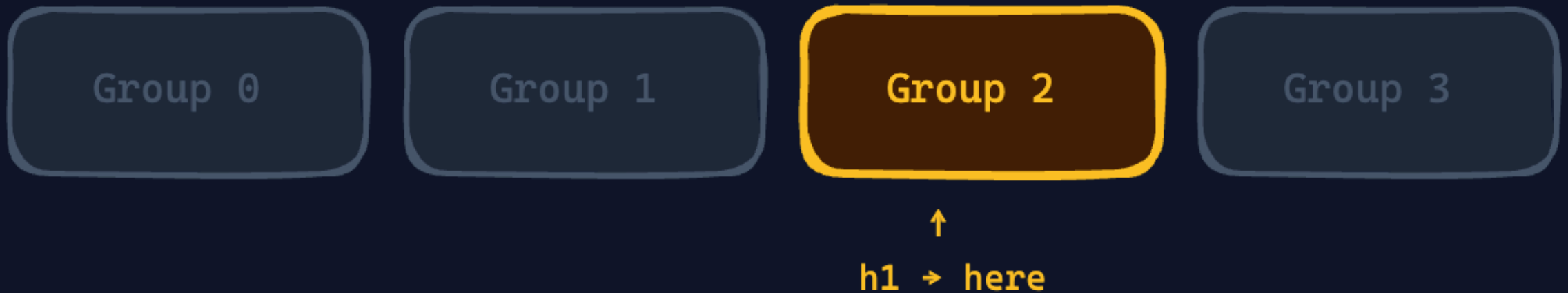
global_depth = 2 → use top 2 bits of hash



Insert: Select Group

Step 2: h1 selects group

```
group = h1 & (num_groups - 1)
```

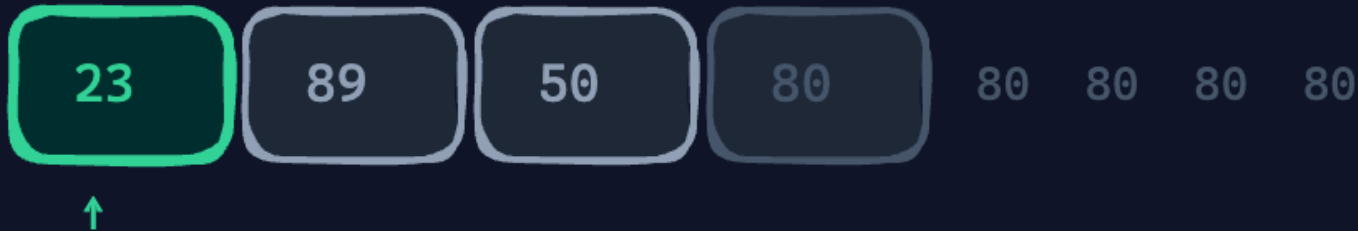


Insert: Match h2

Step 3: compare h2 against all 8 ctrl bytes

h2 = 0x23

Control word:



match! → check full key in slot 0

Key matches → update value | Key differs → not a duplicate

Insert: Find Free Slot

Step 4: find free slot → insert

Key not found → look for empty slot (0x80)



first free → write $h2 + K|V$

After insert:



Group full? → probe next group (triangular probing)

Probing: Triangular

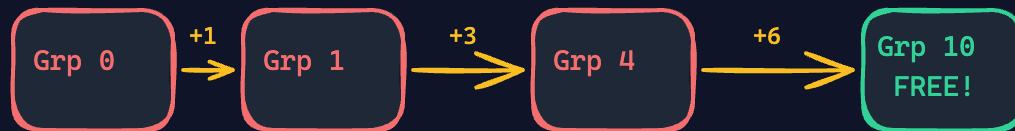
Probing: what happens when the group is

Linear probing



+1, +1, +1 ... clustering problem

Triangular probing (Go)



offset += 1, 3, 6, 10, 15 ... (triangular numbers)

Visits ALL groups when num_groups = 2^M

$$\text{offset}(i) = i \times (i + 1) / 2 \quad (\text{mod num_groups})$$

Tombstones

Tombstones in Open Addressing

1. Insert A, B, C ($h_1 \rightarrow$ Group 0)

[A] [B] [C] [] [] [] [] []

2. Delete B — can't just clear it!

[A] [] [C] [] [] [] [] []

x lookup(C) stops at empty B → miss!

3. Correct: leave a tombstone (T)

[A] [T] [C] [] [] [] [] []

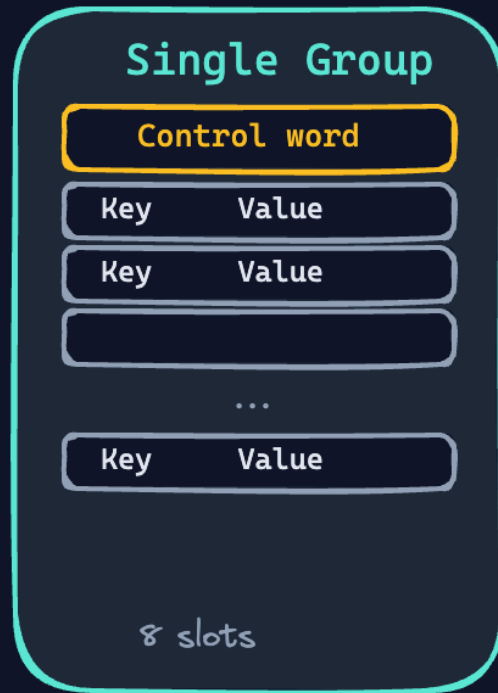
✓ lookup(C) skips T → finds C

ctrl byte: 0x00–0x7F = occupied (h2)
0x80 = empty 0xFE = tombstone

Too many tombstones ($\geq 10\%$) → prune before growing

Small Map Optimization

Small Map: $\text{hint} \leq 8$ or no hint



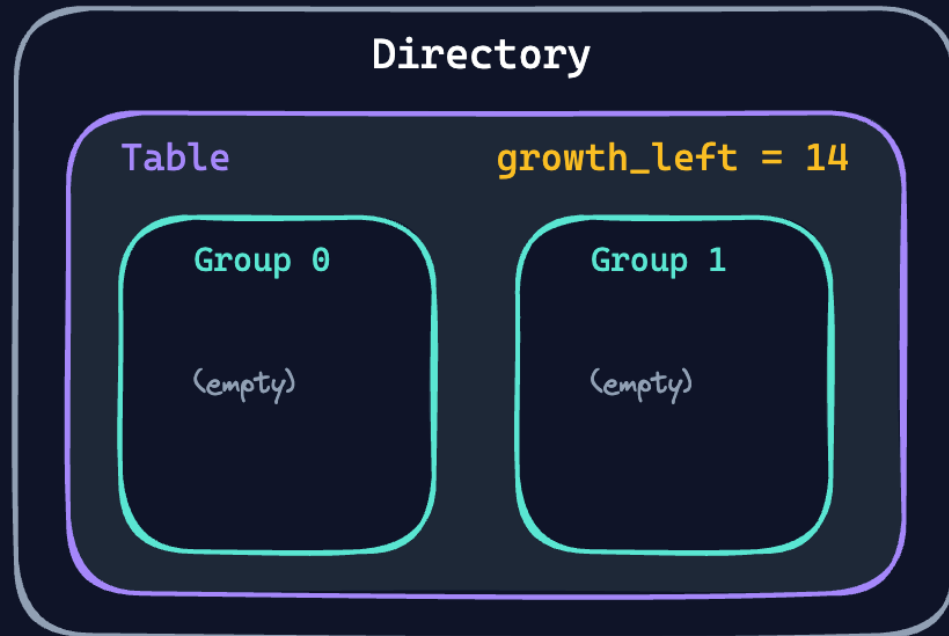
- X No Directory
- X No Table wrapper
- X No tombstones
- ✓ Just one group, inline

On 9th insert →
evolves to full Directory + Table structure

Growth: How It Starts

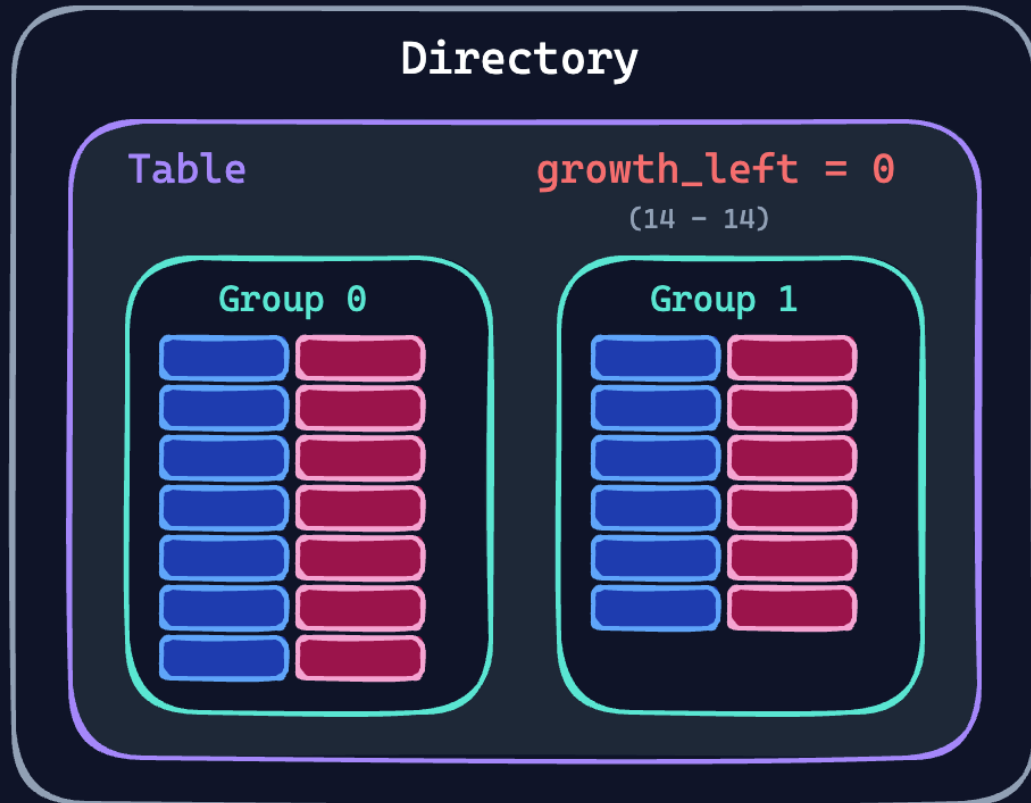
```
make(map[string]string, 10)    hint > 8
```

```
num_groups = 2^M  
load_factor = 87.5% (7/8)  
2^1 = 2 groups    2 × 7 = 14 max
```



*growth_left = how many more
elements this table can accept*

Growth: Filling Up

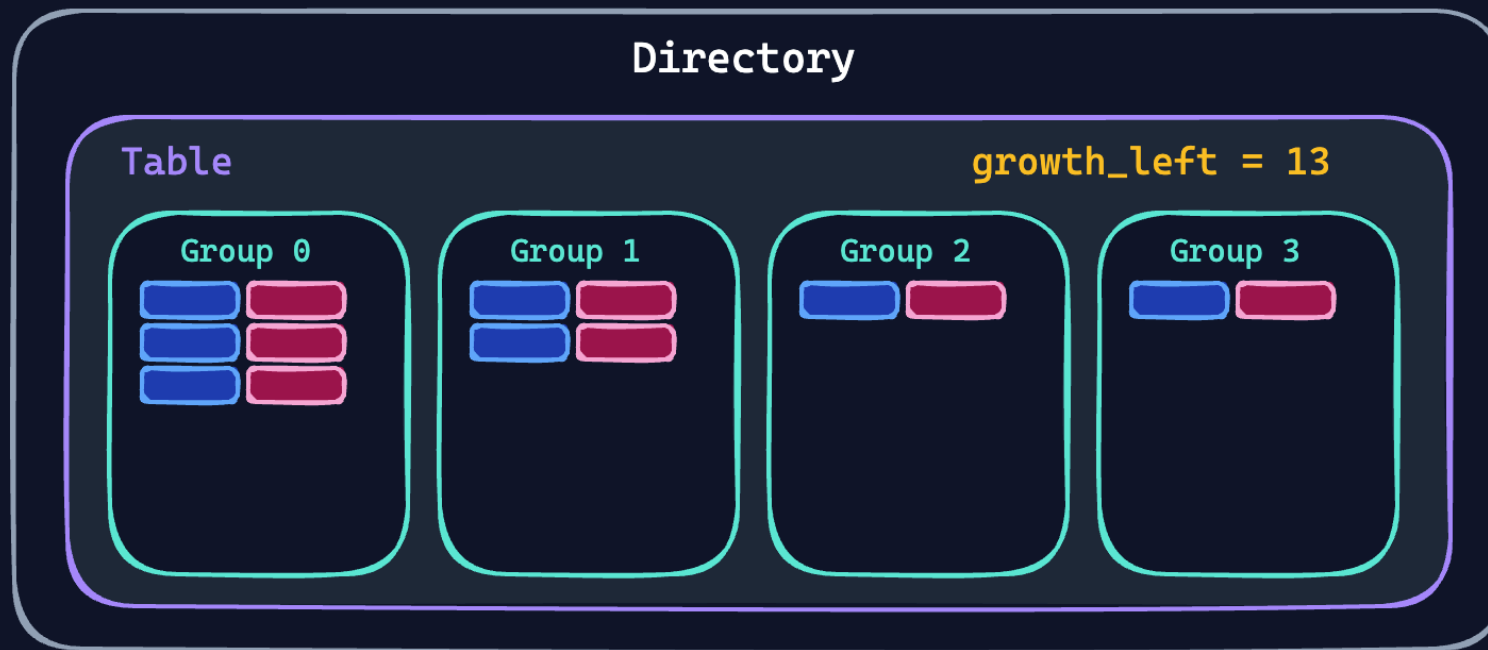


14 elements inserted
growth_left = 0

Next insert → must grow

Growth: Doubling Groups

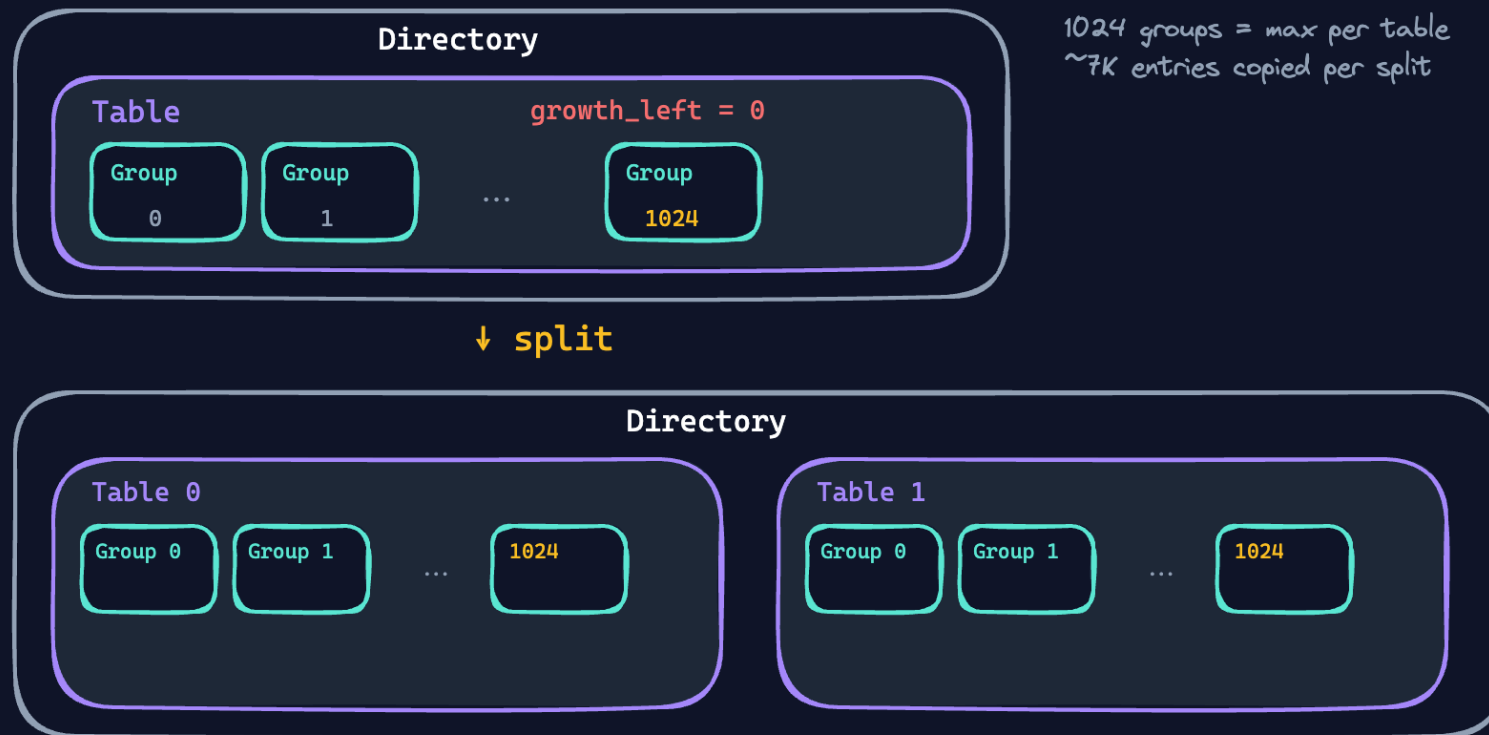
`new growth_left = 4 × 7 - 14 - 1 = 13`



Continues: 4 → 8 → 16 → 32 → ... → 1024 groups

Growth: The Split at 1024

`1024 groups + growth_left = 0 → SPLIT`



Growth: global_depth

```
num_tables = 2^global_depth
```

global_depth = 0 → 1 table

global_depth = 1 → 2 tables

global_depth = 2 → 4 tables

global_depth = 3 → 8 tables

Each table grows independently

Directory

global_depth = 3

Table 0

Table 1

Table 2

Table 3

Table 4

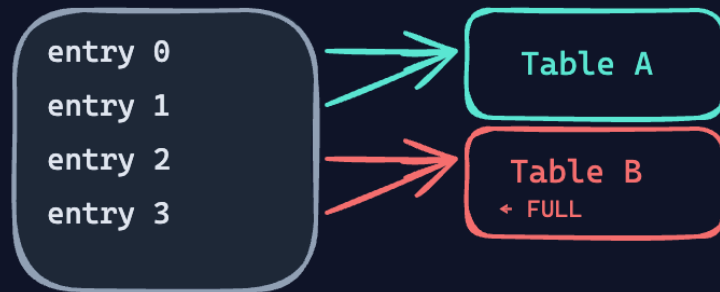
Table 5

Table 6

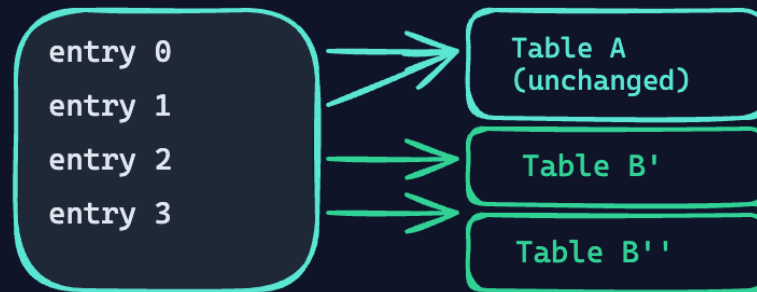
Table 7

Growth: Extendible Hashing

Before split:



After split:



local_depth vs global_depth

`local_depth` = snapshot of `global_depth` when table was created

if `local_depth < global_depth`

→ table has spare directory entries → no directory growth needed

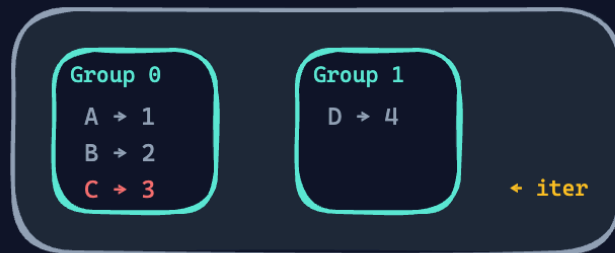
Table A: `local_depth = 1 < global_depth = 2`

Table B', B'': `local_depth = 2 = global_depth`

Iteration + Mutation

Iteration during mutation

Old Table (visit order)

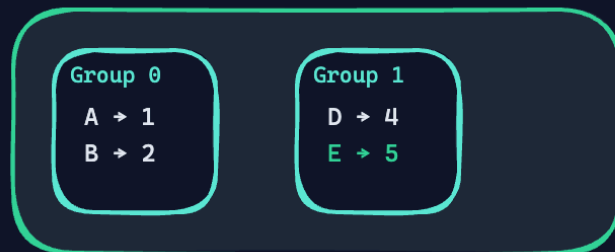


Old table → determines iteration order
New table → reads current values

C deleted from new, E added to new
Iterator continues from old table position

↓ delete(C), insert(E)

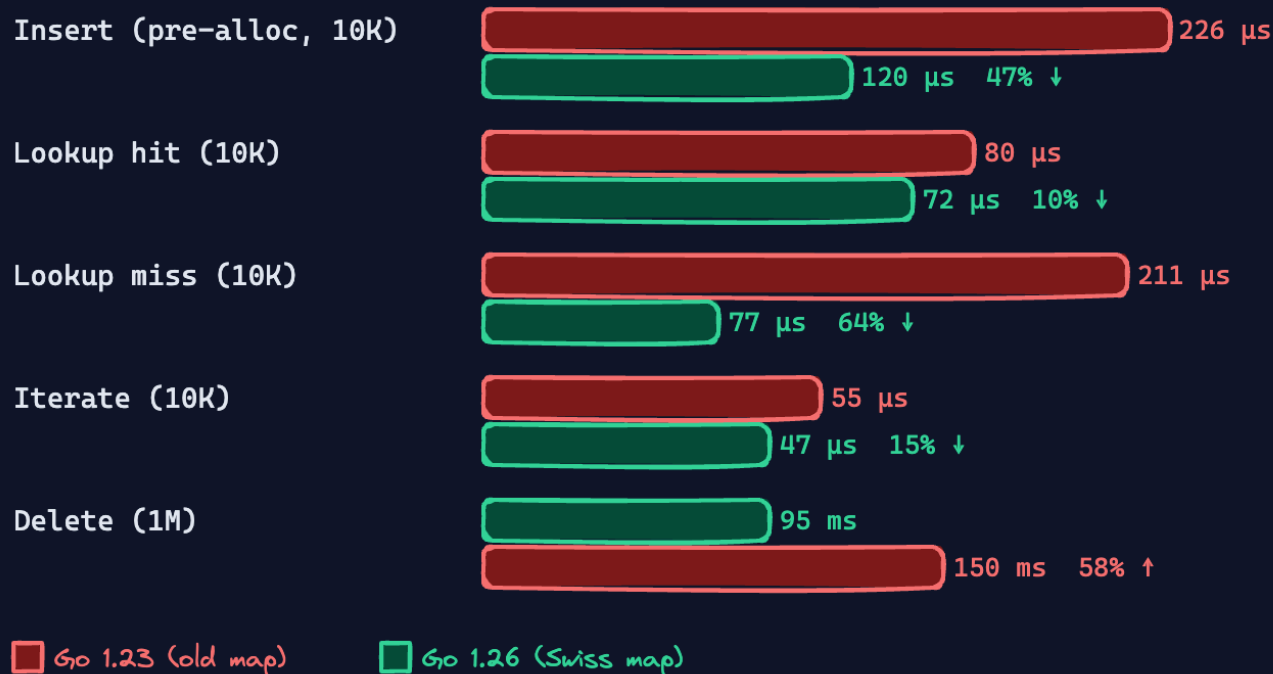
New Table (current values)



Benchmarks: Go 1.23 vs Go 1.24

Benchmarks: Go 1.23 (old map) vs Go 1.24 (Swiss map)

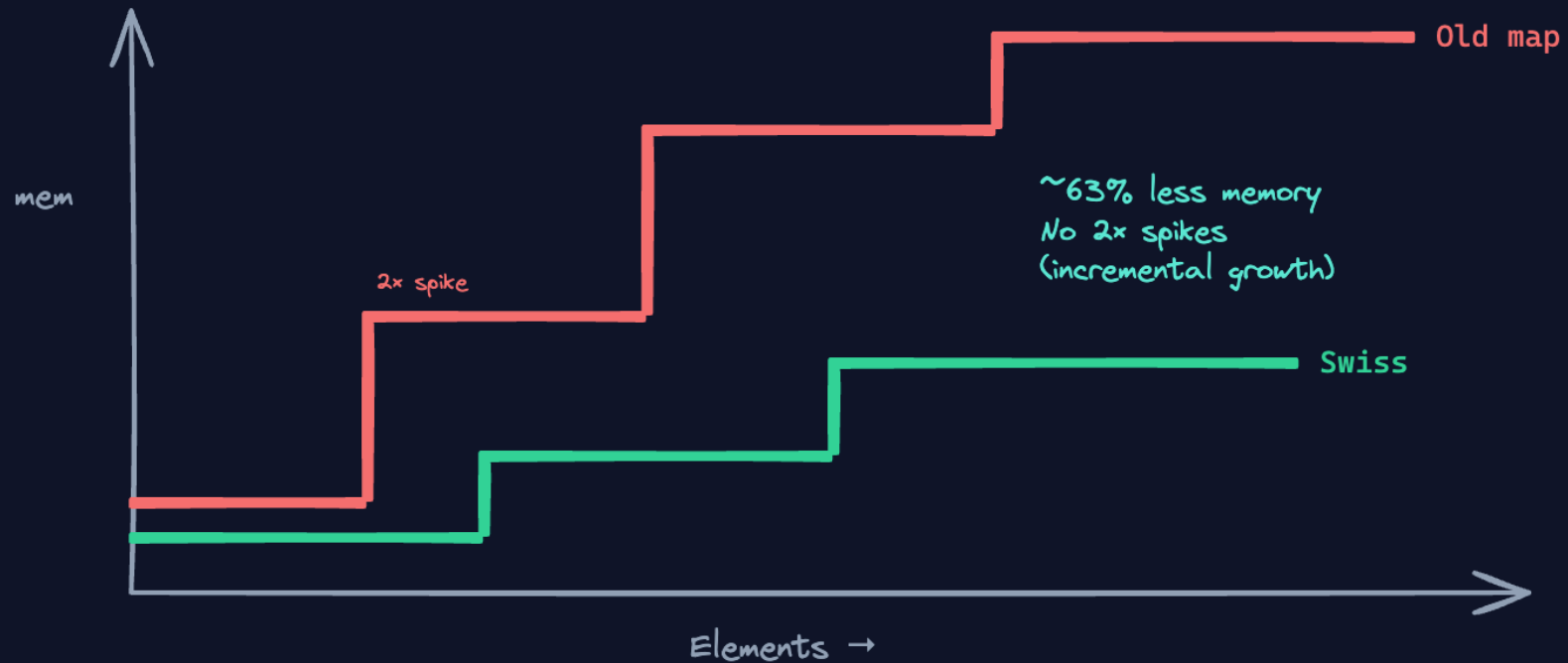
Apple M4 Pro • ARM64 • portable path (no SIMD)



Delete slower — **tombstone bookkeeping**.

Benchmarks: Memory

Memory: Old Map vs Swiss Map



Struct: Old vs New

Old map (hmap)

```
count    int
B         uint8    // log2(buckets)
hash0    uint32    // seed
buckets  *[]bmap
oldbuckets *[]bmap // during growth
-----
bmap (bucket):
tophash  [8]uint8
keys     [8]keyType
values   [8]valType
    ↑ separate arrays!
overflow *bmap
    ↑ pointer chasing!
```

Swiss map (Map)

```
used      uint64
seed      uintptr
dirPtr    *[]table
globalDepth uint8
-----
table:
growthLeft uint16
localDepth uint8
groups     []group
-----
group:
ctrl      uint64    // 8 metadata bytes
slots    [8]struct{K,V}
    ↑ K+V together, no overflow!
```

Summary: Old vs New

	Old Map	Swiss Map
Collision	Chaining (overflow)	Open addressing
Matching	Sequential (1 by 1)	Parallel (all 8)
K-V layout	Separate arrays	Together
Growth	Evacuation	Extendible hashing
Load factor	81%	87.5%
Cache	Pointer chasing	Contiguous

Credits

C++: Matt Kulukundis, Sam Benzaquen, Alkis Evlogimenos, Jeff Dean

Go: Michael Pratt, Peter Mattis, YunHao Zhang, PJ Malloy

Sources: GopherCon (Pratt), Grafana (Boreham), Yandex Cloud, Oleg Kozyrev, Pure Storage, Skill Issue

Questions?

TPF Labs, Trading Solutions — tpf-labs.com

